.NET Developer Interview Preparation Guide

(Covers 2–3 yrs & 3–5 yrs roles — OOPs, C#, .NET Core, ASP.NET Core, EF Core, SQL Server, Design Patterns, DI, Middleware, SDLC)

1. OOPs (Object-Oriented Programming) Concepts

Q1. What are the four pillars of OOP?

Encapsulation — Binding data and methods together into a single unit (class) and restricting direct access to internal state using access modifiers (private, protected).
Abstraction — Hiding implementation details and exposing only relevant functionality (via abstract classes/interfaces).
Inheritance — A class (child) acquiring properties/behavior of another class (parent), enabling code reuse.
Polymorphism — Ability of an object to take many forms; same method behaves differently based on the object calling it.

Q2. Difference between Abstraction and Encapsulation? Abstraction is about *hiding complexity* (design-level, "what to show"), Encapsulation is about *hiding data* (implementation-level, "how to protect it"). Abstraction is achieved via interfaces/abstract classes; Encapsulation via access modifiers and properties.

Q3. Abstract class vs Interface?

Abstract Class	Interface
Can have implementation + abstract methods	Traditionally only method signatures (C# 8+ allows default implementation)
Supports fields, constructors	No instance fields/constructors
Single inheritance only	A class can implement multiple interfaces
Use when classes share a common base with some shared code	Use when unrelated classes must guarantee certain behavior (a "contract")

Q4. Types of Polymorphism?

Compile-time (Static) Polymorphism — Method Overloading, Operator Overloading.
Run-time (Dynamic) Polymorphism — Method Overriding using `virtual`/`override`, achieved through inheritance.

Q5. What is Method Overloading vs Overriding?

Overloading: Same method name, different signature (parameters), same class, resolved at compile time.
Overriding: Redefining a base class's `virtual`/`abstract` method in a derived class using `override`, resolved at runtime (via vtable).

Q6. Can we overload the `Main` method? Can we overload static methods? Yes to both — overloading only depends on the method signature, not on `static` or being an entry point.

Q7. What is Constructor and types of constructors in C#? A constructor initializes an object. Types: Default, Parameterized, Copy constructor (manual, C# has no built-in copy ctor), Static constructor (runs once, initializes static members, no access modifier/parameters), Private constructor (restricts instantiation — used in Singleton).

Q8. What is `virtual`, `override`, `new`, and `sealed`?

`virtual` — allows a method to be overridden in derived class.
`override` — provides new implementation of a virtual/abstract method.
`new` — hides (not overrides) a base class member (method-hiding).
`sealed` — prevents further inheritance of a class, or prevents further overriding of an overridden method.

Q9. What are SOLID principles?

Single Responsibility — a class should have only one reason to change.
Open/Closed — open for extension, closed for modification.
Liskov Substitution — derived classes must be substitutable for their base classes without breaking behavior.
Interface Segregation — many small specific interfaces are better than one large general interface.
Dependency Inversion — depend on abstractions, not concretions (this is the theoretical basis for DI).

Q10. What is Composition vs Inheritance? Why "favor composition over inheritance"? Inheritance creates tight "is-a" coupling; changes in base class can break derived classes (fragile base class problem). Composition ("has-a") builds objects using other objects as members, giving more flexibility and loose coupling — generally preferred for code reuse in modern design.

Q11. What is an interface with default implementation (C# 8+)? Interfaces can now have method bodies (default implementations) so existing implementers don't break when new members are added — useful for API versioning.

2. C# Core Language Concepts

Q12. Value types vs Reference types? Value types (`struct`, `int`, `bool`, `enum`) are stored on the **stack** and hold data directly; copying creates an independent copy. Reference types (`class`, `string`, `array`, `object`) are stored on the **heap**; the variable holds a reference/pointer, so copying copies the reference, not the data.

Q13. What is Boxing and Unboxing? Boxing: converting a value type to `object` (stored on heap). Unboxing: extracting the value type back from the object. Both have performance overhead and should be minimized in performance-critical code.

Q14. What are Delegates? Multicast delegates? Func/Action/Predicate? A delegate is a type-safe function pointer — holds a reference to a method with a matching signature. Multicast delegate can point to multiple methods (invoked in sequence using `+=`).

`Action<T>` — no return value.
`Func<T,TResult>` — returns a value.
`Predicate<T>` — returns bool.

Q15. What are Events, and how do they differ from Delegates? An event is a wrapper around a delegate that restricts external code from invoking it directly or reassigning it (`=`); only the declaring class can raise it. Events follow the publisher-subscriber pattern.

Q16. What is the difference between `const`, `readonly`, and `static`?

`const` — compile-time constant, must be assigned at declaration, implicitly static.
`readonly` — runtime constant, can be assigned in constructor, per-instance unless also static.
`static` — belongs to the type, not instance; shared across all objects.

Q17. Exception Handling — `try/catch/finally`, custom exceptions, `throw` vs `throw ex`. `finally` always executes (cleanup code) regardless of exception. `throw;` preserves the original stack trace; `throw ex;` resets it — always prefer `throw;` when re-throwing. Custom exceptions should inherit from `Exception` and typically end with the suffix "Exception".

Q18. What is `async`/`await`? What problem does it solve? They enable asynchronous, non-blocking programming. `async` marks a method as asynchronous; `await` pauses execution until the awaited `Task` completes, without blocking the calling thread — crucial for scalable I/O-bound operations (DB calls, API calls) in web apps.

Q19. `Task` **vs** `Thread` **vs** `async/await`**?** `Thread` is a low-level OS construct (expensive). `Task` represents an asynchronous operation (can run on thread pool). `async/await` is syntactic sugar over `Task` for writing asynchronous code that reads like synchronous code.

Q20. What is Deadlock in async code, and how to avoid it? Commonly caused by blocking async code with `.Result` or `.Wait()` on the UI/ASP.NET context thread. Avoid by using `await` consistently ("async all the way") and `ConfigureAwait(false)` in library code.

Q21. What are Generics? Why use them? Generics allow defining classes/methods with a placeholder type (`List<T>`, `Dictionary<TKey,TValue>`) providing type safety and avoiding boxing/unboxing, while enabling code reuse across types.

Q22. What is LINQ? Deferred vs Immediate execution? LINQ (Language Integrated Query) allows querying collections/databases using SQL-like syntax in C#. Methods like `Where`, `Select` use **deferred execution** (query runs only when enumerated, e.g., via `foreach` or `.ToList()`); methods like `.Count()`, `.ToList()`, `.First()` trigger **immediate execution**.

Q23. `IEnumerable` **vs** `IQueryable`**?** `IEnumerable` executes queries in-memory (client-side) and is best for LINQ-to-Objects. `IQueryable` builds an expression tree and executes the query on the data source (e.g., translates to SQL in EF), useful for remote/DB queries — more efficient for filtering before data is loaded into memory.

Q24. Difference between `List`**,** `Array`**,** `Dictionary`**,** `HashSet`**?**

`Array` — fixed size, fastest, strongly typed.
`List<T>` — dynamic-size array, allows duplicates, ordered.
`Dictionary<TKey,TValue>` — key-value pairs, fast O(1) lookup by key.
`HashSet<T>` — unique unordered elements, fast lookups, no duplicates.

Q25. What is the difference between `string` **and** `StringBuilder`**?** `string` is immutable — every modification creates a new object (expensive in loops). `StringBuilder` is mutable, ideal for repeated concatenation in loops.

Q26. What are Extension Methods? Static methods that appear to "extend" an existing type without modifying its source code, defined in a static class with `this` keyword on the first parameter. Example: LINQ methods themselves are extension methods on `IEnumerable<T>`.

Q27. Nullable types & Null-conditional operators? `int?` (Nullable<int>) allows value types to hold null. `?.` (null-conditional) and `??` (null-coalescing) and `??=` help write concise null-safe code.

Q28. What is Reflection? Ability to inspect metadata of assemblies/types/members at runtime and dynamically create instances/invoke methods — used in frameworks, serializers, dependency injection containers, ORMs.

3. .NET Core / ASP.NET Core Concepts

Q29. What is .NET Core / .NET 5+ and how is it different from .NET Framework? .NET Core (now unified as ".NET" from .NET 5 onward) is **cross-platform** (Windows/Linux/macOS), **open-source**, modular (via NuGet packages), and higher-performing than the Windows-only .NET Framework. It supports side-by-side versioning and is the current path forward (.NET Framework is legacy/maintenance mode).

Q30. Explain the ASP.NET Core Middleware Pipeline. Middleware are components assembled into a pipeline to handle HTTP requests/responses. Each middleware can process a request, pass it to the next component via `next()`, and process the response on the way back — forming a chain of responsibility. Configured in `Program.cs` (or `Startup.cs` in older versions) using `app.Use...()` methods, and **order matters** (e.g., Exception handling → HTTPS redirection → Static files → Routing → Authentication → Authorization → Endpoints).

```csharp
app.UseExceptionHandler("/Error");
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

Q31. How do you write Custom Middleware?

```csharp
```

```
public class RequestLoggingMiddleware
{
  private readonly RequestDelegate _next;
  public RequestLoggingMiddleware(RequestDelegate next) => _next = next;

  public async Task InvokeAsync(HttpContext context)
  {
    // logic before request
    Console.WriteLine($"Request: {context.Request.Path}");
    await _next(context); // pass to next middleware
    // logic after response
  }
}
// Registration: app.UseMiddleware<RequestLoggingMiddleware>();
```

Q32. `app.Use` vs `app.Run` vs `app.Map`?

`Use` — adds middleware to pipeline, can call `next()` to continue.
`Run` — terminal middleware, doesn't call `next()`, short-circuits pipeline.
`Map` — branches the pipeline based on request path.

Q33. What is Dependency Injection (DI)? Why is it built into .NET Core? DI is a design pattern implementing Inversion of Control (IoC) — dependencies are provided to a class from outside rather than the class creating them itself. This promotes loose coupling, testability (easy to mock), and adheres to the Dependency Inversion Principle. ASP.NET Core has a **built-in IoC container** (`IServiceCollection`) — no need for third-party libraries like Autofac/Ninject (though they can still be used).

Q34. What are Service Lifetimes in .NET Core DI? (Very commonly asked)

```csharp
builder.Services.AddTransient<IMyService, MyService>();
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddSingleton<ICacheService, CacheService>();
```

Q35. What happens if you inject a Scoped service into a Singleton? (Captive Dependency) This causes a **captive dependency** problem — the scoped service gets "captured" by the singleton and effectively becomes a singleton itself, potentially causing stale data or threading issues (e.g., a DbContext living far longer than intended). ASP.NET Core throws an exception in validation-enabled environments to catch this.

Q36. Constructor Injection vs Property Injection vs Method Injection? Constructor Injection is the most common and recommended in .NET Core (dependencies passed via constructor, ensuring the object is always in a valid state). Property Injection sets dependencies via public properties (used when dependency is optional). Method Injection passes the dependency directly into a method that needs it.

Q37. What is `IServiceCollection` vs `IServiceProvider`? `IServiceCollection` is used to **register** services (build-time). `IServiceProvider` is used to **resolve** (retrieve) registered services at runtime.

Q38. Explain `Program.cs` in .NET 6+ (Minimal Hosting Model) vs old `Startup.cs`. .NET 6+ merged `Startup.cs` and `Program.cs` into a single top-level statements file using `WebApplicationBuilder` — `builder.Services` replaces `ConfigureServices`, and `app.Use...` replaces `Configure`. It reduces boilerplate while functioning identically underneath.

Q39. What is `appsettings.json` and Configuration in .NET Core? A hierarchical JSON-based configuration file, environment-specific (`appsettings.Development.json`, `appsettings.Production.json`). Accessed via `IConfiguration`, and can be strongly typed using the **Options pattern** (`IOptions<T>`, `IOptionsSnapshot<T>`, `IOptionsMonitor<T>`).

Q40. What are Filters in ASP.NET Core MVC? Filters run at specific stages of the request pipeline within MVC:

Authorization Filters — run first, check access.
Resource Filters — wrap the rest of the pipeline (caching).
Action Filters — run before/after an action method executes.
Exception Filters — handle unhandled exceptions.
Result Filters — run before/after the action result is executed.

Q41. What is Routing? Convention-based vs Attribute Routing? Routing maps incoming URLs to controller actions. Convention-based routing is defined centrally (`pattern: "{controller}/{action}/{id?}"`). Attribute routing uses `[Route]`, `[HttpGet]`, etc., directly on controllers/actions — preferred for Web APIs for clarity.

Q42. MVC vs Web API — are they merged in .NET Core? Yes — in ASP.NET Core, `Controller` (MVC) and `ApiController` (Web API) are unified under a single `Microsoft.AspNetCore.Mvc` framework; both can coexist in the same project.

Q43. What is Model Binding and Model Validation? Model Binding automatically maps incoming HTTP request data (query strings, form data, route values, JSON body) to action method parameters/objects. Model Validation uses Data Annotations (`[Required]`, `[StringLength]`, `[Range]`) or FluentValidation, checked via `ModelState.IsValid`.

Q44. What is Kestrel? Kestrel is the lightweight, cross-platform web server built into ASP.NET Core, often used behind a reverse proxy like IIS, Nginx, or Apache in production for added security/load balancing features.

Q45. What is CORS and how do you enable it? Cross-Origin Resource Sharing — a browser security mechanism that restricts requests from a different origin (domain/port). Enabled in .NET Core via:

```
csharp
builder.Services.AddCors(options => options.AddPolicy("AllowAll",
    p => p.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader()));
app.UseCors("AllowAll");
```

Q46. Authentication vs Authorization in .NET Core? Authentication verifies **who** the user is (login, JWT tokens, cookies, Identity). Authorization determines **what** the authenticated user can access (roles, policies, claims-based).

Q47. What is JWT and how does token-based auth work in Web API? JSON Web Token is a compact, self-contained token (Header.Payload.Signature) used for stateless authentication — the server issues a signed token on login, and the client sends it in the `Authorization: Bearer <token>` header on subsequent requests; the server validates the signature without needing session storage.

Q48. What is Caching in .NET Core? In-memory vs Distributed? `IMemoryCache` — caches data in the web server's memory (fine for single-server apps). `IDistributedCache` (Redis, SQL Server) — shared cache across multiple server instances, essential for scaled/load-balanced apps.

Q49. What is the difference between `appsettings` environment variables and Secrets Manager? Environment-specific config layering: `appsettings.json` → `appsettings.{Environment}.json` → environment variables → command-line args (later ones override earlier). `dotnet user-secrets` is used in development to avoid committing sensitive data (connection strings, API keys) to source control.

4. Entity Framework Core

Q50. Code-First vs Database-First approach? Code-First: define C# entity classes; EF generates the database schema via Migrations. Database-First: scaffold entity classes from an existing database using `Scaffold-DbContext`. Code-First is more common in greenfield projects; DB-First suits legacy databases.

Q51. What are Migrations in EF Core? Migrations track incremental changes to the model and apply them to the database schema in a versioned manner (`Add-Migration`, `Update-Database`) without losing existing data.

Q52. Lazy Loading vs Eager Loading vs Explicit Loading?

Lazy Loading — related data loaded automatically the first time it's accessed (needs virtual navigation properties + proxies enabled); risk of N+1 query problem.
Eager Loading — related data loaded upfront using `.Include()`/`.ThenInclude()`.
Explicit Loading — related data loaded manually and explicitly via `.Entry().Reference()/.Collection().Load()`.

Q53. What is Change Tracking / `DbContext` states? EF Core tracks entity states: `Added`, `Modified`, `Deleted`, `Unchanged`, `Detached`. `SaveChanges()` generates SQL based on tracked state. `AsNoTracking()` is used for read-only queries to improve performance by skipping change tracking.

Q54. `DbContext` lifetime in ASP.NET Core — why is it typically Scoped? `DbContext` is lightweight and not thread-safe, meant to represent a single unit of work — registering it as **Scoped** ensures one instance per HTTP request, avoiding concurrency issues and memory bloat.

Q55. What is the N+1 query problem, and how do you avoid it? Occurs when a query fetches a list of entities, then executes a separate query per item to fetch related data (via lazy loading), causing many round-trips to the DB. Solved via Eager Loading (`Include`) to fetch everything in one/few queries.

5. SQL Server / Database Concepts

Q57. What are the types of SQL Joins?

INNER JOIN — matching rows in both tables.
LEFT (OUTER) JOIN — all rows from left + matched rows from right (NULL if no match).
RIGHT (OUTER) JOIN — all rows from right + matched from left.
FULL OUTER JOIN — all rows from both, matched where possible.
CROSS JOIN — Cartesian product of both tables.
SELF JOIN — table joined with itself.

Q58. What is Normalization? Explain 1NF, 2NF, 3NF. Normalization organizes data to reduce redundancy and improve integrity.

1NF — atomic values, no repeating groups.
2NF — 1NF + no partial dependency (non-key attributes depend on the whole primary key).
3NF — 2NF + no transitive dependency (non-key attributes depend only on the primary key, not on other non-key attributes).

Q59. Clustered vs Non-Clustered Index? Clustered Index physically sorts and stores the table's data rows in order of the index key — only **one per table** (usually the primary key). **Non-Clustered Index** is a separate structure with pointers back to the data rows — a table can have **multiple**, useful for speeding up queries on frequently filtered columns.

Q60. What is a Primary Key vs Foreign Key vs Unique Key? Primary Key uniquely identifies each row (no NULLs, one per table). Foreign Key enforces referential integrity by linking to another table's primary key. Unique Key ensures uniqueness but allows one NULL, and a table can have multiple.

Q61. What are ACID properties?

Atomicity — transaction is all-or-nothing.
Consistency — DB moves from one valid state to another.
Isolation — concurrent transactions don't interfere with each other.
Durability — once committed, changes persist even after a system failure.

Q62. What is a Stored Procedure? Advantages? A precompiled set of SQL statements stored in the database. Advantages: better performance (execution plan caching), reduced network traffic, reusability, and improved security (can grant execute permission without granting table access).

Q63. Stored Procedure vs Function? Functions must return a value and can be used in SELECT statements; they cannot perform DML operations (INSERT/UPDATE/DELETE) or use `TRY-CATCH`. Stored Procedures can perform DML, don't have to return a value, and support transaction handling.

Q64. What are Triggers? Special stored procedures that automatically execute in response to DML events (INSERT/UPDATE/DELETE) on a table — used for auditing, enforcing business rules, or maintaining derived data.

Q65. What is a Transaction, and how do you implement it in SQL/C#? A transaction groups multiple operations into a single unit of work that either fully succeeds (`COMMIT`) or fully fails (`ROLLBACK`). In SQL: `BEGIN TRAN ... COMMIT/ROLLBACK`. In EF Core: `using var transaction = await context.Database.BeginTransactionAsync();`.

Q66. `WHERE` vs `HAVING`? `WHERE` filters rows before grouping/aggregation; `HAVING` filters groups after `GROUP BY`/aggregation is applied.

Q67. What is a CTE (Common Table Expression)? A temporary named result set defined with `WITH` that can be referenced within a single query — improves readability and enables recursive queries.

Q68. `DELETE` vs `TRUNCATE` vs `DROP`? `DELETE` removes rows (can use WHERE, logged, can be rolled back, triggers fire). `TRUNCATE` removes all rows quickly (minimal logging, resets identity, can't use WHERE). `DROP` removes the entire table structure permanently.

Q69. What is Query Optimization / how do you improve a slow query? Check execution plan, add proper indexes on filtered/joined columns, avoid `SELECT *`, avoid functions on indexed columns in WHERE clauses, use proper JOINs instead of subqueries where possible, update statistics, and avoid cursors when set-based operations suffice.

Q70. What is Deadlock in SQL Server, and how to prevent it? Occurs when two transactions hold locks the other needs, causing a circular wait. Prevented by accessing tables/resources in a consistent order, keeping transactions short, and using appropriate isolation levels.

6. Design Patterns (commonly asked)

Q71. What is Singleton Pattern? How is it implemented thread-safely? Ensures only one instance of a class exists application-wide, with a global access point. Thread-safe implementation uses `Lazy<T>` or a lock:

```csharp
public sealed class Singleton
{
  private static readonly Lazy<Singleton> _instance = new(() => new Singleton());
  public static Singleton Instance => _instance.Value;
  private Singleton() { }
}
```

(In .NET Core DI, registering a service as `AddSingleton` achieves this automatically.)

Q72. What is the Repository Pattern, and why use it? Abstracts data access logic behind an interface (`IRepository<T>`), decoupling business logic from data-access/ORM specifics — improves testability (can mock the repository) and centralizes query logic.

Q73. What is Unit of Work Pattern? Coordinates multiple repository operations within a single transaction, ensuring all changes are committed or rolled back together — commonly paired with the Repository pattern, and closely mirrors what `DbContext` already does in EF Core.

Q74. Factory Pattern vs Abstract Factory? Factory Pattern provides a method to create objects without exposing instantiation logic to the client (one product family). Abstract Factory provides an interface to create *families* of related objects without specifying their concrete classes.

Q75. What is the Strategy Pattern? Defines a family of interchangeable algorithms/behaviors, encapsulated as separate classes implementing a common interface, allowing behavior to be selected/swapped at runtime.

7. WCF / WPF Basics (JD-mentioned, may come up briefly)

Q76. WCF vs Web API — when to use which? WCF supports multiple protocols (HTTP, TCP, MSMQ, Named Pipes) and multiple message formats (SOAP, REST) — heavier and more configuration-driven. Web API is lightweight, HTTP-only, REST-focused, and is the modern standard for building APIs (WCF is largely legacy now).

Q77. What is WPF and what is XAML/MVVM? WPF (Windows Presentation Foundation) is a UI framework for desktop apps using XAML for declarative UI design. MVVM (Model-View-ViewModel) is the standard architectural pattern used with WPF — View binds to ViewModel

properties/commands via Data Binding, keeping UI and logic separated for testability.

8. SDLC / Agile / Process (Additional Responsibilities section)

Q78. Explain Agile methodology and Scrum ceremonies. Agile is an iterative approach delivering working software in short cycles (sprints). Scrum ceremonies: Sprint Planning, Daily Stand-up, Sprint Review, Sprint Retrospective. Key roles: Product Owner, Scrum Master, Development Team.

Q79. What is Unit Testing? What frameworks have you used? Testing individual units/methods in isolation, typically using **xUnit**, **NUnit**, or **MSTest**, often combined with **Moq** for mocking dependencies. Good unit tests follow the AAA pattern: Arrange, Act, Assert.

Q80. What is CI/CD? Continuous Integration — automatically building/testing code on every commit. Continuous Deployment/Delivery — automatically deploying tested code to staging/production. Common tools: Azure DevOps, Jenkins, GitHub Actions.

9. Scenario-Based Questions (frequently asked in real interviews)

Q81. "We have a slow API endpoint — how would you debug and fix it?" Approach: Check logs/Application Insights for time breakdown → check the DB query (execution plan, missing indexes, N+1 issue) → check for synchronous blocking calls that should be async → check for unnecessary data being loaded (use projection/`Select`) → add caching if data is read-heavy and doesn't change often → load test to confirm improvement.

Q82. "How would you design a system where multiple users can book the same seat, and you must prevent double-booking?" Discuss: Database transactions with proper isolation level, optimistic concurrency (`RowVersion`/`Timestamp` column) vs pessimistic locking, or a unique constraint on (SeatId, ShowId) at the DB level to guarantee data integrity even under concurrent requests.

Q83. "Your Singleton service is throwing stale/incorrect data intermittently in production — what could be wrong?" Likely a captive dependency (a Scoped/Transient service like DbContext injected into it) causing shared/corrupted state across requests, or unsynchronized mutable state accessed by multiple threads without locks.

Q84. "How do you handle configuration differences between Dev, QA, and Production environments?" Using `appsettings.{Environment}.json` layering, environment variables for secrets, `ASPNETCORE_ENVIRONMENT` variable to switch, and Azure App Configuration/Key Vault for centralized/secure settings in production.

Q85. "How would you version an API that's already being used by clients?" Discuss options: URL versioning (`/api/v1/...`), Query string versioning, Header versioning, or Media-type versioning — using the `Microsoft.AspNetCore.Mvc.Versioning` package, while maintaining backward compatibility for existing consumers.

10. Quick-Fire Round (rapid recall — good for last-minute revision)

Question	One-line Answer
Stack vs Heap	Stack = value types/local vars, fast, LIFO; Heap = reference types, managed by GC
Garbage Collection generations	Gen 0 (short-lived), Gen 1 (medium), Gen 2 (long-lived/static)
`is` vs `as`	`is` checks type (bool); `as` casts and returns null if it fails (no exception)
`Equals()` vs `==`	`Equals` checks value equality (can be overridden); `==` checks reference by default for classes
`IDisposable` / `using`	Used for deterministic cleanup of unmanaged resources (DB connections, file handles)
Partial class	Splits a class definition across multiple files, compiled as one
Anonymous types	`var x = new { Name = "A" }` — compiler-generated read-only type
Tuple vs `ValueTuple`	`ValueTuple` (`(int, string)`) is a lightweight struct-based alternative to `Tuple` class
Record type (C# 9+)	Immutable reference type with built-in value equality, great for DTOs
`HttpClient` best practice	Use `IHttpClientFactory` instead of `new HttpClient()` to avoid socket exhaustion

11. Interview Tips

For 2–3 yrs role: expect deeper focus on C# fundamentals, OOPs, basic EF/SQL, and hands-on coding (write a small class/LINQ query live).

For 3–5 yrs role: expect system design-lite questions, DI lifetime scenarios, performance/debugging scenarios, and "why" questions (why Scoped for DbContext, why async, why Repository pattern) — they're testing judgment, not just definitions.
Always explain **"why"**, not just "what" — interviewers probe reasoning behind design choices.
Be ready to **write code on a whiteboard/shared doc**: e.g., reverse a string, singleton implementation, LINQ query on a list of objects, a simple middleware.
Prepare 2–3 questions to ask the interviewer (team structure, tech stack, code review practices) — shows engagement.
Revisit your own resume/projects — be ready to explain any design decision you claim to have made.

Good luck! Revise Sections 3 (ASP.NET Core/DI/Middleware) and 5 (SQL) most thoroughly since they are the highest-frequency topics for this JD.